

Production Readiness Audit Report

Phone Shop POS -- Flutter Desktop

Repository: hussainshahzad350/phone_shop_pos

Audit Date: 2026-05-16 | DB Version: 6 | App Version: 0.1.0+1 | ~12,000 LOC

SECTION 1 -- ARCHITECTURE AUDIT

Overall Assessment: STRONG FOUNDATION WITH SPECIFIC GAPS

The project follows clean architecture consistently across all modules. Domain/data/presentation layering is properly enforced. Repository contracts live in domain/repositories/ and are implemented in data/repositories/. Service classes handle business logic independently of the DB layer. Riverpod providers act as the DI mechanism. Result<T> sealed class pattern is applied consistently.

Architecture Issues

Severity	Issue	Location
MEDIUM	InvoiceNumberGenerator class is dead code -- never imported or called. Real atomic invoice	sales/services/invoice_number_gener
MEDIUM	ModulePlaceholderService and ModulePlaceholderWidget exist but are never used. All routes	core/widgets/module_placeholder_scr
MEDIUM	Duplicate model files in inventory scaffolding as it has its own customer_model.dart and	modules/inventory/data/models/
LOW	Supplier models that have diverged (missing notes, invoice fields) from canonical one.	modules/customers/
LOW	DashboardService and salesCustomerService bypass the repository pattern and talk directly to	dashboard/services/dashboard_servic

SECTION 2 -- DATABASE AUDIT

Overall Assessment: SOLID WITH TWO CRITICAL DATA INTEGRITY ISSUES

Schema design is thoughtful. WAL mode enabled, foreign keys on, busy timeout configured, migrations versioned from v1 to v6. Atomic invoice sequence via invoice_sequences table is correct. Sale and purchase transactions wrap all writes atomically. Cost price snapshot saved to sale_items at sale time (v2 migration).

Database Issues

Severity	Issue	Location
CRITICAL	imei2 has NO database-level UNIQUE constraint. Only imei1 is unique-indexed. Two records	migration_service.dart --
CRITICAL	could store the same imei2 value through direct DB edit or race condition. Inventory unit_cost overwritten on every purchase. Daily profit queries use ist.unit_cost (live)	serializedStock table sqlite_purchase_repository.dart:265,
HIGH	Profit calculations in reports (and others) to historical profit reports. Sales only change cost query	sales_report_service.dart:35
	(snapshot) but DailySalesReport and Dashboard use ist.unit_cost (live). Same date range	

shows different profit in different tabs.

		sales_report_service.dart:35,
HIGH	Customer balance report groups by customer_name (TEXT), not customer_id. If two	inventory_report_service.dart:173 profit_report_service.dart:26
MEDIUM	customers share a name, their balances are merged into one row. Pre-transaction IMEI uniqueness check runs before the transaction opens. Another write	

could insert the same IMEI in the window between check and insert (low risk in single-user desktop, not strictly safe).

69

Phone Shop POS -- Production Readiness Audit Report

MEDIUM	No max_quantity sales enforcement. Schema has CHECK constraint but nothing prevents	migration_service.dart,
LOW	selling_products table has max_quantity constraint but has no module, repository, service, or screen.	migration_sales_repository.dart
LOW	Users table has password_hash, role, is_active fields but authentication is completely	migration_service.dart:users

SECTION 3 -- BUSINESS LOGIC AUDIT

Overall Assessment: CORE WORKFLOWS SOLID; OPERATIONAL WORKFLOWS MISSING

Create-sale and create-purchase pipelines are complete, validated, and transactionally safe. IMEI tracking lifecycle (purchase intake -> in_stock -> sold) is correctly implemented. Inventory and stock queries work correctly.

Missing Workflows -- Production Blockers

Severity	Missing Feature	Impact
CRITICAL	No "Collect Payment" / Update Paid Amount feature. Once a credit	Customer receivables management is impossible
CRITICAL	No Sale History / Invoice List screen. After a sale is saved, there is no sale is saved, there is no way to record subsequent payment.	Cashiers cannot review any past sale
HIGH	No Purchase history screen. Purchase screen verify from sales list of	Inventory audit trail inaccessible in normal use
HIGH	Customer balances grow unboundedly. No Sale/Returns/Refund Review. Supplier receipts saved it is permanent.	Returns are impossible without manual DB editing
HIGH	No mechanism to return a phone or reverse an accessory sale. No Stock Adjustment feature. No screen to manually adjust inventory	Inventory is unreconcilable after physical counts
MEDIUM	No authentication / login. Any person who opens the app has full quantity. The adjustInventoryQuantity method exists in repository but	Zero access control
LOW	No Expense tracking, including table up/return No Expense tracking, including table up/return but no UI, so profit	Misleading profit figures
	is not exposed in UI.	

SECTION 4 -- DESKTOP UX AUDIT

Overall Assessment: WELL-DESIGNED KEYBOARD FLOW; THREE CRITICAL USABILITY GAPS

Shortcut architecture is solid: AppShortcutManager registers F1/F2/F5/F10/Ctrl+F/S//R/B globally. Screens listen to appShortcutEventBusProvider and respond correctly. Search debounce is 150ms. Focus-after-add works correctly on IMEI selection.

UX Issues

Severity	Issue	Location
CRITICAL	Windows runner opens at 1280x720 but app constant defines 1366x768 as minimum. No	windows/runner/main.cpp:29,
HIGH	"Next Page" button on Reports never disables. When a page returns fewer rows than window_manager package -- minimum size NOT enforced. On a 1366x768 screen the sales	reports_screen.dart:113-127 app_constants.dart:6-7
HIGH	PageSpaceWidget did click/loseTextfield empty results withing Controller. On widget rebuild	sales/presentation/widgets/totals_pan
MEDIUM	UI will clip. The type of receipt is accessible from every page through the Open button. It reprint a receipt	settings_screen.dart
LOW	Other buttons to help on navigation. Settings Settings Widget can info snackbar instead of triggering backup	

directly.

app_shortcut_manager.dart:140-146

LOW

AppSidebar does not highlight active route if path does not exactly match the route string.

desktop_navigation_shell.dart:36

SECTION 5 -- PERFORMANCE AUDIT

Overall Assessment: GOOD INDEXING; TWO QUERY DESIGN RISKS

Indexes are well-planned and cover all critical join and filter columns. Queries use parametrized LIMIT clauses. QueryDiagnostics.trace identifies slow queries in debug mode (> 120ms threshold).

Performance Issues

Severity	Issue	Location
HIGH	DashboardService.getDashboardKpis executes two complex 5-table JOIN queries on every	dashboard_service.dart:16-65
HIGH	getStockRows with one caching WALs of two complex queries with a keyset pagination (only	sqlite_inventory_repository.dart:178-2
MEDIUM	ReportOfSalesOptionsProvider 15ms default keyset optionsProvider run unconditional	report_providers.dart:200-225
LOW	Full table scans for findWALFilter and provides with basic access by large esp	sales_query_providers.dart:33
LOW	QueryDiagnostics.trace is disabled (Debug mode) Slow query detection never runs in	query_diagnostics.dart:16-22

SECTION 6 -- CODE QUALITY AUDIT

Overall Assessment: ABOVE AVERAGE; SPECIFIC TECHNICAL DEBT

Dart null safety correctly applied throughout. Naming conventions consistent. Result<T> pattern used correctly. Code is readable with appropriately scoped widgets. Single-quoted strings and package imports enforced via linter.

Code Quality Issues

Severity	Issue	Location
HIGH	InvoiceNumberGenerator is dead code -- never imported or used. Future developers may	invoice_number_generator.dart
HIGH	ModulePlaceholderScreen and AppPlaceholderWidget are dead code -- no route uses them.	module_placeholder_screen.dart,
MEDIUM	TotalsPanelWidget has no TextEditingController for discount/tax fields, causing visual	totals_panel_widget.dart
MEDIUM	SalesBillingScreen is 850+ lines (35.8KB). Handles search, IMEI picker, cart, shortcuts, sale	sales_billing_screen.dart
MEDIUM	Complete customer and supplier model files in inventory/data/models/ are diverged from	inventory/data/models/
LOW	Major constant 2000s.paginateThreshold in AppDataTable without a named constant or	desktop_components.dart:164
LOW	SaleItem model missing from Map factory. Querying sale items back from DB (needed for sale	sale_item_model.dart

SECTION 7 -- SECURITY & DATA SAFETY AUDIT

Overall Assessment: BACKUP/RESTORE IS SAFE; AUTH AND IMEI2 ARE GAPS

Staged backup/restore pipeline is well-implemented: WAL checkpoint before copy, temp file staging, integrity check of staged file, atomic rename, rollback on failure. These are good production-level practices.

Security Issues

Severity	Issue	Location
----------	-------	----------

Phone Shop POS -- Production Readiness Audit Report

CRITICAL	No authentication. Any user who opens the app has full operator access including backup,	Entire app
HIGH	imei2, and all DB-level uniqueness constraint. Only application-level check during purchase.	migration_service.dart
HIGH	Direct DB edit/insert/update/delete can silently create duplicate IMEI providers	database_backup_service.dart:125,
MEDIUM	has backup path metadata as plain App Store in the database after successful backup	settings_backup.dart:229-2

SECTION 8 -- MVP COMPLETENESS AUDIT

Module	Status	Notes
Dashboard	COMPLETE	KPIs, recent sales, low stock. Real queries, not mocks.
Sales -- Create Sale	COMPLETE	Cart, IMEI picker, totals, customer select, payment method, F10 save.
Sales -- History/Lookup	MISSING	No past invoice browsing, no receipt reprint from history.
Sales -- Payment Update	MISSING	No "Collect Payment" for credit sales.
Sales -- Void/Return	MISSING	No reversal, refund, or void workflow.
Purchases -- Create	COMPLETE	IMEI entry, bulk IMEI, quantity stock update, supplier link.
Purchases -- History	MISSING	No past purchase browsing.
Inventory -- View/Filter	COMPLETE	Status filter, IMEI search, type filter, summary cards.
Inventory -- Stock Adjust	MISSING	No manual adjustment, no write-off.
Inventory -- IMEI Edit	MISSING	Cannot correct an IMEI entry after intake.
Master Data	COMPLETE	Products, brands, customers, suppliers -- CRUD operational.
Reports -- Daily Sales	COMPLETE	With date filters and export.
Reports -- Date Range Sales	COMPLETE	Fully implemented.
Reports -- Profit	PARTIAL	Correct formula but inconsistent with daily sales profit figure.
Reports -- Sold Phones	COMPLETE	Fully implemented.
Reports -- Current Stock	COMPLETE	Fully implemented.
Reports -- Customer Balance	PARTIAL	Groups by name, not ID -- can merge distinct customers with same name.
Reports -- Low Stock	COMPLETE	Fully implemented.
Reports -- Purchase Report	MISSING	No purchase history or COGS-by-supplier report.
Settings / Backup	COMPLETE	One-click backup, restore with rollback, health dashboard.
Authentication	MISSING	Schema present, login flow absent.
Printing	PARTIAL	File spool works; no direct printer driver; queue management in Settings only.
Expenses	MISSING	Schema exists, no UI or service.

SECTION 9 -- FINAL SCORECARD

Area	Score	Rationale
Architecture	7/10	Clean separation and consistent patterns; dead code, duplicate models, and services-bypassing-repo issues prevent a higher score.
Database	6/10	Good schema and migration design; imei2 not unique-constrained, unit_cost overwrite bug, and inconsistent profit calculation are meaningful data integrity risks.
UX	5/10	Good keyboard workflow and shortcut architecture; missing sale history, payment collection, and window size enforcement failure are significant cashier friction points.
Performance	7/10	Solid indexing and debouncing; two expensive unguarded join queries and no production query monitoring.
Maintainability	7/10	Readable code, consistent patterns; one giant screen file, dead code, and duplicate models hurt this score.
Production Readiness	4/10	Core transaction logic is safe; too many critical business workflows are missing for a

real shop to operate on this build alone.

SECTION 10 -- CRITICAL NEXT ACTIONS

1. Highest Priority Data Integrity Fixes (Before Any Live Use)

1. Fix the unit_cost overwrite in `sqlite_purchase_repository.dart:265`. The daily sales profit query must use `si.cost_price` (snapshot), not `ist.unit_cost` (live).
2. Add a partial UNIQUE index on `serialized_stock.imei2` for non-null values via a new v7 migration: `CREATE UNIQUE INDEX IF NOT EXISTS idx_serialized_stock_imei2_unique ON serialized_stock(imei2) WHERE imei2 IS NOT NULL`
3. Fix customer balance report to GROUP BY `s.customer_id` instead of `customer_name` to prevent balance merging for customers with the same name.
4. Align daily sales profit to use `si.cost_price` instead of `ist.unit_cost` -- same formula as `profit_report_service.dart`.
5. Fix `TotalsPanelWidget` to use a `TextEditingController` for discount and tax fields, synchronized with Riverpod state.

2. Dangerous Technical Debt

1. Dead `InvoiceNumberGenerator` class -- delete it or mark clearly as "internal format helper only" to prevent accidental use.
2. Duplicate `customer_model.dart` and `supplier_model.dart` in inventory module -- delete and import from canonical module.
3. `SaleItemModel` missing from `Map` factory -- add it so sale history queries can be built.

3. Production Blockers

1. Payment Collection workflow -- a "Collect Payment" action that updates `sales.paid_amount` must exist before going live.
2. Sale History screen -- cashiers must be able to look up past invoices.
3. Window size enforcement -- add `window_manager` package to enforce 1366x768 minimum, or fix `main.cpp` to start at 1366x768.
4. Authentication -- any multi-employee shop needs login/role separation before real data is entered.
5. Purchase History screen -- needed for audit, supplier disputes, and inventory reconciliation.

4. Missing MVP Items (Next Sprint)

1. Sale void/return with stock reversal
2. Manual stock adjustment screen
3. Purchase history list view
4. Expenses module (UI for the existing schema)
5. Partial payment collection after sale
6. "Next Page" disable condition on Reports when results < `pageSize`
7. Purchase report tab in Reports module

5. Recommended Next Milestone

Before the first real-shop pilot, resolve in order:

1. Fix the four data integrity issues (unit_cost overwrite, imei2 uniqueness, customer balance grouping, profit inconsistency)
2. Add Sale History screen with receipt reprint
3. Add Payment Collection on credit sales
4. Fix window size enforcement
5. Add Purchase History screen

After pilot validation: Authentication / role-based access, Sale void/return, Expenses tracking, Stock adjustment, Automated backup reminders.